ICT159 – Assignment 1

Student ID	32712899
Name	JACK DU BOULAY
Final Mark	

Objectives

This is an individual assignment. It is worth 10% of the total assessment for the unit. The objectives are to:

- Apply top-down design and modular programming to construct an algorithmic solution to a complex problem
- Use a combination of sequence, selection, and iteration constructs.
- Implement design and algorithms in the C programming language.

Note that this assignment must be completed.

This assignment requires you to apply all concepts learnt in the unit from Topics 1 to 4. You must **not** use concepts covered from Topic 5 onwards.

Important note

This is an individual assignment and must be completed by you alone. Any unauthorised collaboration/collusion or use of Generative AI will be subject to investigation and will result in penalties if found to be in breach of the University policy.

Submission Guidelines

Your assignment must be submitted via LMS following the guidelines listed below. Failure to adhere to these guidelines will result in marks being lost or a Fail grade being awarded.

- Create a folder named ICT159_StudentID_Surname_Assignment1. Make sure you replace StudentID with your student ID and Surname with your surname.
- Put the C programs of the assignment into this folder.
- Compress the folder ICT159_StudentID_Surname_Assignment1 using ZIP.
- Answer the questions directly on this Word document (the C programs need to be included separately; see the previous points. Do not paste code into this word document). Once done, convert it into PDF.
- Upload the ZIP file and the PDF to the assignment submission point on the LMS.
- You should keep a copy of all your work. Keep the backup of your work in cloud storage like OneDrive. Losing your work/files (for any reason) a few days before the deadline is not valid ground for a deadline submission extension.

Note that:

- Progress on this assignment must also be demonstrated in-person prior to submission. This progress demonstration is done in the labs leading to the submission.
- You will be required to demonstrate and explain your solution in-person after the submission.
- If you do not defend/demonstrate in-person, no marks will be given — resulting in a mark of 0.

You must use modular design and modular programming throughout the assignment. Your C implementation must match your design and algorithm.

Failure to use modular programming will result in a straight 0.

Assignment Question

You are asked to write a modular solution (algorithm and C program) that will accept as input an amount of money (in cents) in a certain currency and return the optimal number of coins in that currency.

The amount of money entered in is in cents and should be an integer value in the range of 1 to 95 inclusive.

The program should work as follows; First, it will ask the user to select a currency among the following three:

1) US$, which has four types of coins: 50, 25, 10 and 1 cents

2) AU$, which has four types of coins: 50, 20, 10 and 5 cents

3) Euro, which has four types of coins: 20, 10, 5 and 1 cents

Note that these coins are randomly chosen just for the purpose of this exercise and may not correspond to the actual reality.

The program then should ask the user to input an amount of money in cents as a number between 1 to 95 inclusive. If the chosen currency is AU$, your program **should ensure that the input value is a multiple of 5**.

Your program should then return the minimum number of coins (in the selected currency) that sum up into the input number.

The solution should aim to give as much of the higher valued coins as possible. For example, a poor solution for an input of AU 30 cents would give six 5 cent coins. A correct solution would give a 20-cent coin and a 10-cent coin.

After each output, the user should be asked whether they wish to continue (i.e., enter another amount) or exit/terminate the program.

Your solution (algorithm and program) should be designed using a modular approach. This requires the submission of a structure chart, a high-level algorithm, and subsequent decompositions of each step (i.e. the algorithms of the different modules).

Important Note: For this problem, the principles of code reuse, low coupling and high cohesion, covered in Week 4, are particularly important: A significant number of marks are allocated to these aspects of your design.

- You should attempt to design your solution such that it consists of a relatively small number of modules (functions) but still meets the modular design best practice requirements of this unit.
- In particular, strive to have one module that can be reused (called repeatedly) to solve the coin calculation problem. If you find that you have developed a large number of modules where each performs a similar task, or that you have a lot of instructions that are repeated in one module, then attempt to analyse your design to generalise the logic so that you have just one general version of the module. See the **DRY principle** https://www.geeksforgeeks.org/software-engineering/dont-repeat-yourselfdry-in-software-development/. You really do not want to write unnecessary code as you must spend more time debugging and that is a waste of your valuable time.
- Be mindful of the cohesion exhibited by each module. If you have a module that is doing more than one task (i.e. demonstrating low cohesion), then you should re-design it to have high cohesion.

What to submit

1. Assumptions (5%)

All assumptions made other than those stated in the question that you make about the problem. There will virtually always be assumptions you are implicitly making so think about this very carefully. However, your assumptions cannot contradict the assignment specification. Also be careful that you do not put in unnecessary assumptions.

- Program Input assumptions:
  - Expecting one integer input for selection of currency types.
  - Expecting one integer input for change amount.
  - Expecting one integer input for selection to retry/quit program.
  - Where all three integer inputs may be able to use the same module with varying min/max ranges.

- Program Processing assumptions:
  - Validation of currency type int selection (3 types of currencies (1-3) inclusive)
  - Validation of change amount so that it's within range (1-95 (US/EU) or 5-95 (AU) inclusive)
    - (5-95) for AU is justified as the assignment specifies to only accept multiples of 5.
  - Validation of retry/quit program int selection (Retry / Quit)
  - If any user inputs are considered invalid, re-prompt the user to try again, this can be done using a do-while loop.
  - Sorting of coins should be done with div or mod to find the number of largest to smallest coins.
  - Main function should contain a do-while loop to allow the user to select an option to quit or retry.

- Program Output assumptions:
  - Display a prompt for user to enter currency type.
  - Display a prompt for user to enter change amount.
  - Display an output for the number of coins for each coin value.
  - Display a prompt for user to enter a value to retry/quit program

- Any other assumptions:
  - 4 separate integers will be required to store the values of coins for each currency
    - This is because we are **not** allowed to use arrays for this assignment
  - 4 separate integers will be required to store each number of coins AFTER making calculations
  - 2 integers will be required for keeping track of the user's currency choice and their change amount.

2. Structure Chart (10%)

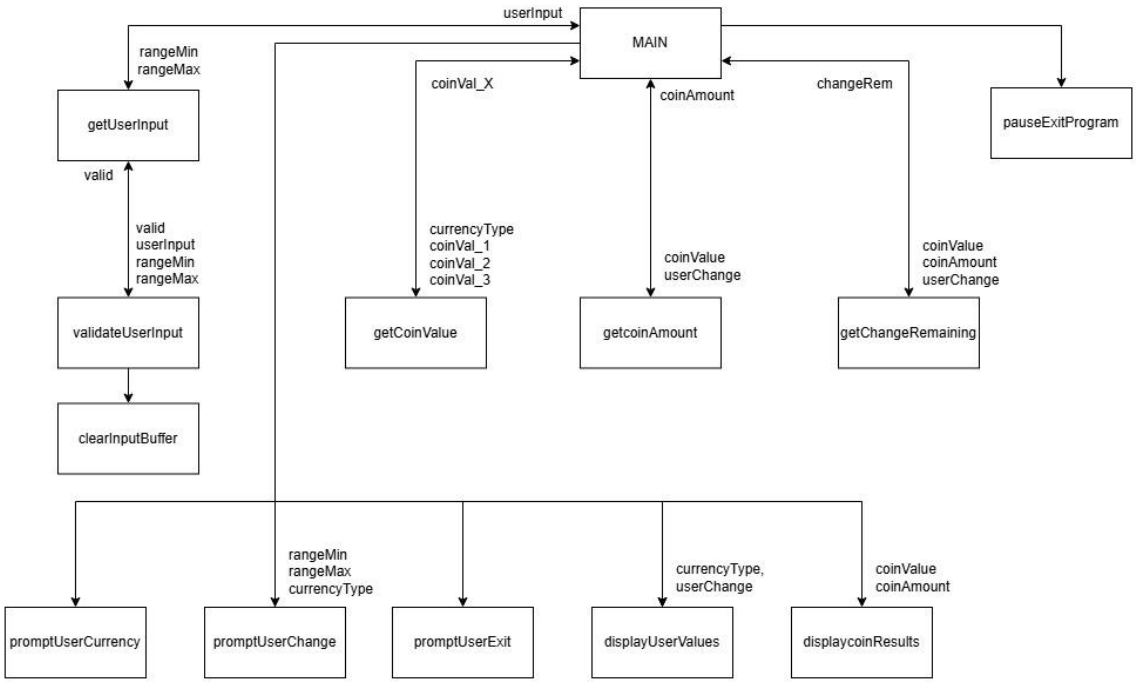
Provide the structure chart of your program, which should show the different modules that compose your program as well as the parameter passing.

Note that structure chart is different from flowchart. Structure chart is about the modular decomposition of the problem; please refer to the learning materials of Topic 4 (Modular Programming). Only the approach shown in Topic 4 can be used.

Also, your algorithm / program should match this structure chart otherwise marks will be deducted.

You can use any tool to draw your structure chart, e.g., Visio or PowerPoint (among others). Just make sure it is clear and easy to read.

Make sure to give all parameters meaningful names. Don't include the parameter types in the structure chart, just the names. Include the parameter types in the table below.



Fill in the following table which provides more detail about each parameter. Add rows as necessary.

Module Name ***(as shown on structure chart)***	**Parameter name** ***(as shown on structure chart)***	**Parameter type** ***(e.g. int, float, .etc)***	**Parameter direction** ***(e.g. in, out, in/out)***	**Purpose of Module/Parameter**	**Single Responsibility (Y/N)**
getUserInt	rangeMin	Int	in	Keeps user's integer input within minimum range.	Yes
	rangeMax	Int	in	Keep's user's integer input within maximum range.	Yes
	userInput	Int	out	Returns a valid integer value back to main	Yes
validateUserInt	Valid	Int	In/out	Value used to display the correct error messages during validation. If errors are found then return valid = 0, else if no errors found then return valid = 1;	yes
	userInput	Int	in	Value used for IF statement comparisons/calculations to display the appropriate error message.	yes
	rangeMin	Int	in	Read only value used for IF statement comparisons/calculations.	yes
	rangeMax	Int	In	Read only value used for IF statement comparisons.	yes
clearInputBuffer	-	-	-	Clears the input buffer if the user inserted an invalid value	yes
promptUserCurrency	-	-	-	Displays currency menu	yes
promptUserChange	rangeMin	Int	in	Read only value showing minimum range and comparison.	yes
	rangeMax	Int	in	Read only value showing maximum range	yes
	currencyType	Int	in	Read only value used for a comparison condition to display a message if AUD was selected.	yes
getCoinValue	currencyType	Int	In	Read only value, used for comparison logic.	Yes
	coinVal_1	Int	In	Never read, but is returned when a comparison condition is met	Yes
	coinVal_2	Int	In	Never read, but is returned when a comparison condition is met	Yes
	coinVal_3	Int	In	Never read, but is returned when a comparison condition is met	Yes
	coinVal_x	Int	out	The "x" is one of the coinVal_ values for which condition was met.	Yes
getCoinAmount	coinValue	Int	in	Read only value, used as the divisor for a simple division calculation	yes
	userChange	Int	in	Read only value, used as the dividend for a simple division calculation	yes
	coinAmount	Int	out	The calculation value result, used to get the maximum number of coins of that coin value.	yes
getChangeRemaining	coinValue	Int	In	Read only value used for a multiplication calculation	yes
	coinAmount	Int	In	Read only value used for a multiplication calculation	yes
	userChange	Int	in	Read only value used for a subtraction calculation.	yes
	changeRem	Int	out	The calculation value result, value is used returned to userChange	yes
displayUserValues	currencyType	int	in	Read only value, used for comparison condition. Condition will print out which currency type the user selected.	yes
	userChange	Int	in	Read only value, displaysDisplays what the user had entered as their change amount	yes
displayCoinResults	coinValue	Int	in	Read only, prints the coin value.	yes
	coinAmount	Int	in	Read only, print number of coins	yes
promptUserExit	-	-	-	Displays the exit menu	yes
pauseExitProgram	-	-	-	Displays final instruction. Pauses program awaiting user to hit enter to quit.	yes

3. Algorithm (20%)

Provide your algorithms written using pseudo-code and adhering to the conventions required in the unit. Algorithms not following the conventions specified in this unit would get no marks.

Your algorithm should be presented at an appropriate level of detail sufficient to be easily implemented.

You need to provide the high-level algorithm along with algorithms of your modules (i.e., the algorithm of each module) as appropriate to the question.

Important Notes

- Algorithms that look as if the code was written first and then word processed to look like an algorithm would receive no marks.
- You must start by the algorithms first and then convert them into code
- Your algorithms should match the structure chart you provided in the previous question for any mark to be allocated.
- For marks to be allocated, algorithms should be written following the pseudocode style we saw in the unit (lectures and lab sessions)

High level (main) algorithm

```
-    // Initialize coin variables
-    int coinValue_1, coinValue_2, coinValue_3, coinValue_4;
-    int coinAmount_1, coinAmount_2, coinAmount_3, coinAmount_4;
-
-    // Program variables
-    int userCurrencyType  ← 0;
-    int userChange  ←  0;
-    int quitProgram ←  0;
-
-    do{
        o    // Pick currency
        o    promptUserCurrency(); // Display a selection menu to the user for picking a currency
        o    userCurrencyType ← get a user int in range: ({1}, {3});
        o
        o    // Get the coin values based on the user currency type selected
        o    coinValue_1  ←  getCoinValue(userCurrencyType, {50}, {50}, {20});  // (USD value, AUD value, EUR value)
        o    coinValue_2  ←   getCoinValue(userCurrencyType, {25}, {20}, {10});
        o    coinValue_3  ←   getCoinValue(userCurrencyType, {10}, {10}, {5});
        o    coinValue_4  ←  getCoinValue(userCurrencyType, {1}, {5}, {1});
        o
        o     // Get the change amount
        o    promptUserChange(coinValue_4, {95}); // Display the range for entering next integer value amount
        o    userChange ← getUserInt(coinValue_4, {95});
        o
        o    // Display the user their inputted values before altering userChange
        o    promptUserValues(currencyType, userChange);
        o
        o    // Calculate number of coin then subtract the value from userChange
        o    coinAmount_1  ← getCoinAmount(coinValue_1, userChange);
        o    userChange  ← getChangeRemaining(coinValue_1, coinAmount_1, userChange);
        o    coinAmount_2  ← getCoinAmount(coinValue_2, userChange);
        o    userChange  ← getChangeRemaining(coinValue_2, coinAmount_2, userChange);
        o    coinAmount_3  ← getCoinAmount(coinValue_3, userChange);
        o    userChange  ← getChangeRemaining(coinValue_3, coinAmount_3, userChange);
        o    coinAmount_4  ← getCoinAmount(coinValue_4, userChange);
        o
```

```
        o    // Display the coin value and the number of coins
        o    displayCoinResults(coinValue_1, coinAmount_1);
        o    displayCoinResults(coinValue_2, coinAmount_2);
        o    displayCoinResults(coinValue_3, coinAmount_3);
        o    displayCoinResults(coinValue_4, coinAmount_4);
        o
        o    // Display the user the exit menu
        o    promptUserExit();
        o    exitProgram ← getUserInt({1}, {2});
-    } while (exitProgram != {2});
-    // Display prompt and await user to hit enter to quit
-    pauseExitProgram();
```

Module getUserInt

```
Input:
        o    Value rangeMin of type Int;
        o    Value rangeMax of type Int;
Output:
        o    Value userInput of type Int;
Algorithm:
-    int userInput ← 0;
-    int valid ← 0;
-    do {
        o    print("Enter a value between {rangeMin}-{rangeMax}: ");
        o    valid ← (userInput ← scan number typed by user); // assigns 1 for true, 0 for false
        o    // Check validity - display appropriate error messages
        o    valid ←validateUserInput(valid, userInput, rangeMin, rangeMax);
        o    if (NOT valid) // Re-prompt user to insert a valid value
                ▪    Print("Please enter a valid value between: {rangeMin}-{rangeMax}");
-    } while (valid == 0)
-    return userInput;
```

Module validateUserInput

```
Inputs:
-    value valid of type int
-    value userInput of type int
-    value rangeMin of type int
-    value rangeMax of type int
Outputs:
-    Integer value of 0 or 1 depending on conditions
Algorithm:
-    If (NOT valid)
        o    Print ("User inserted a non-integer value");
        o    clearInputBuffer();
        o    return valid ← 0;
-    If (valid AND getChar() != '\n')
        o    Print("Did not enter a valid integer");
        o    clearInputBuffer();
        o    return valid ← 0;
-    if (valid AND userInput NOT in range)
        o    Print("Out of range");
        o    return valid ← 0;
```

```
-    if (valid AND (userInput % rangeMin != 0)) // AUD edge cases only
          o    Print("Change amount must be multiple of 5");
          o    return valid ← 0;
-    Return valid;
```

Module clearInputBuffer

```
Input:
          o    none
Output:
          o    none
Algorithm:
-    while(getchar() != '\n');
return;
```

Module getCoinValue

```
Input:
          o    value currencyType of type int
          o    Value coinVal_1 of type int // USD value
          o    Value coinVal_2 of type int // AUD value
          o    Value coinVal_3 of type int // EUR value
Output:
          o    Value of type int: coinVal_1 OR coinVal_2 OR coinVal_3
Algorithm:
-    If ( currencyType == [1]) {
          o    Return coinVal_1;
-    }
-    Else if (currencyType == [2]) {
          o    Return coinVal_2;
-    }
-    Else {
          o    Return coinVal_3;
-    }
```

Module getCoinAmount

```
Input:
          o    value coinValue of type int
          o    Value userChange of type int
Output:
          o    Value coinAmount
Algorithm:
-    Int coinAmount  ←  0;
-    coinAmount  ←  userChange / coinValue;
-    return coinAmount;
```

Module getChangeRemaining

```
Input:
          o    value coinValue of type int
          o    value coinAmount of type int
          o    Value userChange of type int
Output:
          o    Value changeRem
Algorithm:
-    Int changeRem  ←  0;
-    changeRem  ←  userChange - (coinValue * coinAmount);
-    return  changeRem;
```

Module promptUserCurrency

```
Input:
        o   none
Output:
        o   none
Algorithm:
-   print("Currency choice menu");
-   print("US Coins: {50, 25, 10, 1}");
-   print("AU Coins: {50, 20, 10, 5}");
-   print("EU Coins: {20, 10,  5, 1}");
-   return;
```

Module promptUserChange

```
Input:
        o   Value rangeMin of type Int;
        o   Value rangeMax of type Int;
        o   Value currencyType of type int;
Output:
        o   none
Algorithm:
-   print("Enter change value");
-   print("Please enter change value between: {rangeMin}-{rangeMax}");
-   If (currencyType == [2])
        o   Print("Change amount must be a multiple of 5); // Notify user of required multiple (AUD)
-   return;
```

Module promptUserExit

```
Input:
        o   None
Output:
        o   none
Algorithm:
-   print("Retry or quit program");
-   print("[{1}] To retry");
-   print("[{2}] To Quit");
-   return;
```

Module displayUserValues

```
Input:
        o   value currencyType of type int;
        o   Value userChange of type Int;
Output:
        o   none
Algorithm:
-   print("User's inputs");
-   print("Currency type selected: ");
-   if (currencyType == [1])
        o   print("USD");
-   else if (currencyType == [2])
        o   print ("AUD");
-   else
        o    print ("EUR");
-   print("Change value inserted: {userChange}");
-   print("| Value | Amount |");
```

```
-    return;
```

Module displayCoinResults

```
Input:
        o    value coinValue of type int
        o    value coinAmount of type int
Output:
        o    none
Algorithm:
-    // OLD print("Coin values and amounts ");
-    // OLD print(" Coin value: {coinValue}  | Amount: {coinAmount});
-    If (coinValue < {10}){
        O    Print ("|   {coinValie}  |   {coinVamount}  |");
-    }
-    Else {
        O    Print ("|  {coinValie}  |   {coinVamount}  |");
-    }
-    Return;
```

Module pauseExitProgram

```
Input:
        o    None
Output:
        o    none
Algorithm:
-    print("Press enter to quit program…");
-    getChar();
-    return;
```

4. Algorithm testing (10%)

Provide a set of test data in tabular form with expected results and desk check results from **your algorithm**. Each test data must be justified – reason for selecting that data. No marks will be awarded unless justification for each test data is provided.

Add rows to the following table as needed. The table can span more than one page. Each test case tests only one condition for the desk check. There should be no duplicated reasons listed in the second column. (Add as many rows as needed)

Construct the test set carefully. If the user of your program finds a problem with your program, and you have not tested it, you may not get a pass mark for the test table as the test table did not do its job. If you fake/falsify a test, you will get no marks for the entire test table.

Test id	Test description/justification – what is the test for and why this particular test.	Actual data for this test	Expected output	Actual desk check (algorithm) result	outcome – Pass/ Fail
1	Testing lower boundary violation for currency selection. A value of 0 will be entered as it's below the minimum range. Why: A user may accidentally enter the value.	Module: validateUserInput valid = 1 userInput: 0 Rangemin: 1 RangeMax: 3	Print: "User entered a value outside of range" return value 0;	1. If (!valid) → false 2. If (valid AND getChar() != '\n') → false 3. If (valid AND ([0] < 1 OR [0] > 3)) → true Print ("out of range"); Return 0;	Pass
2	Testing upper boundary violation for currency selection. A value of 4 will be entered as it's below the minimum range. Why: A user may accidentally enter the value.	Module: validateUserInput valid = 1 userInput: 4 Rangemin: 1 RangeMax: 3	Print: "User entered a value outside of range" return value 0;	1. If (!valid) → false 2. If (valid AND getChar() != '\n') → false 3. If (valid AND ([4] < 1 OR [4] > 3)) → true Print ("out of range"); Return 0;	Pass
3	Testing data type non-numeric violation for currency selection. A value of 'A' will be entered as it's a char value. Why: A user may accidentally enter the value, checking the if !invalid) branch	Module: validateUserInput valid = 0 userInput: 'A' Rangemin: 1 RangeMax: 3	Print: "User entered a non-integer value" clearInputBuffer() return value 0;	1. If (!valid) → true Print ("User inserted a non-integer value") clearInputbuffer() Return 0	Pass
4	Testing a valid value joined with a non-numeric value for currency selection. A value of '2A' will be entered. Why: A user may accidentally enter the value along with their intentional valid value. Doing so will check the If (valid && getChar() != '\n') branch	Module: validateUserInput valid = 1 userInput: '2A' Rangemin: 1 RangeMax: 3	Print: "User entered an integer with a non-integer value." clearInputBuffer() return value 0;	1. If (!valid) → false 2. If (getChar() != '\n') → true Print ("User entered an integer with a non-integer value."); clearInputBuffer() Return 0	Pass
5	Insert a valid value for AUD change amount that is NOT a multiple of 5 A value of '6' will be entered as it's technically a valid integer value within the range. Why: Testing an edge case scenario where the user may enter a value that is not a multiple of 5	Module: validateUserInput valid = 1 userInput: 6 Rangemin: 5 RangeMax: 95	Print: "Change value must be a multiple of 5" return value 0;	1. If (!valid) → false 2. If (valid AND getChar() != '\n') → false 3. If (valid AND ([6] < 5 OR [6] > 95)) → false 4. If (valid AND ([6] % 5 != 0)) → true Print ("Change value must be a multiple of 5"); Return 0;	Pass

	for AUD.				
6	Testing the lower boundary for AUD change amount. A value of '5' will be entered as it's a valid integer value. Why: Testing best case scenario of a user entering the minimum valid value that will pass the IF branch filter. Doing so should return a value back to main.	Module: getUserInput Rangemin: 5 RangeMax: 95 userInput: 5 valid = 1	Returns userInput value: 5	1 valid ← scanf value (5) = true 2. valid ← validateUserInput (1, 5, 5, 95) = true 3. while(!valid) ← false Return userInput = 5	Pass
7	Testing the upper boundary for AUD change amount. A value of '95' will be entered as it's a valid integer value. Why: Testing best case scenario of a user entering the maximum valid value that will pass the IF branch filter. Doing so should return a value back to main.	Module: getUserInput Rangemin: 5 RangeMax: 95 userInput: 95 valid = 1	Returns userInput value: 95	1. valid ← scanf value (95) = true 2. valid ← validateUserInput (1, 95, 5, 95) = true 3. while(!valid) ← false Return userInput = 95	Pass
8	Test correct value allocation, A currencytype of 2 will be entered for inserting a value for the first coin. Why: Testing a scenario that verifies that the correct value is inserted for the first coin. If that works, then the rest should work.	Module: getCoinValue currencyType: 2 (AUD) coinVal_1: 50 coinVal_2: 50 coinVal_3: 20	Returns coinVal_2 value: 50	1. if (currencyType == 1) ← false 2. If (currencyType == 2) ← true return coinVal_2;	Pass
9	Test getting the correct coin amount. Using currencyType 3, we will use the first coin and find the appropriate amount coins to allocate for the userChange of 89. Why: Because we want the highest number of coins for each coin value from highest to lowest.	Module: getCoinAmount currencyType: 3 (EUR) coinValue: 20 userChange: 89	Returns coinAmount value: 4	1. int coinAmount ← userChange / coinValue; // 89 / 20 = 4 (remainder 9) return coinAmount	Pass
10	Test the remaining change after calculation. using currencyType 3, number of coins 4, a coin value of 20 and user's change of 89. Why: We need to make sure that the correct change is left over after finding subtracting the coin amount multiplied by the coin value.	Module: getChangeRemaining currencyType: 3 (EUR) coinValue: 20 coinAmount: 4 userChange: 89	Returns changeRem value: 9	1. int changeRem ← userChange – (coinValue * coinAmount); //89 – (4 * 20) = 9 return changeRem;	Pass

5. C programs (40%)

Provide the C programs that correspond to the solution. Put all the C program files that you created in the folder ICT159_StudentID_Surname_Assignment1. Make sure you use the code style required in the unit. Do not copy-paste code into this document.

To get any mark for this section, the structure of your C program must match the structure chart you provided, and the code implementation follows the algorithm you provided.

Make sure the code compiles with 0 errors and 0 warnings:

- 10 marks will be deducted if the compiler generates warnings and
- 20 marks will be deducted if it generates errors.

In addition to providing the source code, you must fill up the table below. In the column, "Purpose of the function/module", you will need to ensure that the function/module that you have written does only one thing: **Single responsibility principle**. Find out what is meant by the "Single responsibility principle" and how it relates to the cohesion principle.

High level functions like main() call other functions to get the job done. These other functions can call other functions, etc. The lower-level functions (designed using a structure chart) do single tasks. As an example, a function call sum only does adding items together. This function will not ask for user input and will not do any output of the result to screen.

In the column "Single responsibility? (Y/N)" you need to ensure that the code implementation of the function/subroutine does only one thing, in which case the answer is Y for yes. If you find that the code function/subroutine is doing more than one thing, you must refactor the function/subroutine. Do that by first examining your algorithm and structure chart to determine how to further modularise your design.

Extend the rows in the following table as needed. Functions need to match what is in the structure chart and algorithm. If there are a number of functions in the same file, you write the file name once in the File name column for the first function listed in the table. Successive functions (in the same file) do not need to list the file name repeatedly.

No marks awarded if the table below is not filled.

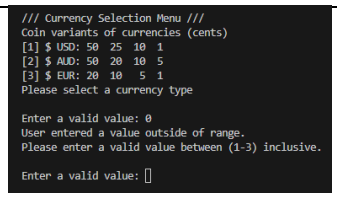
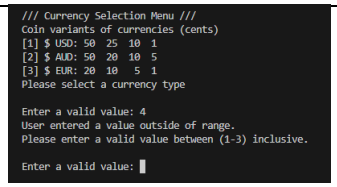
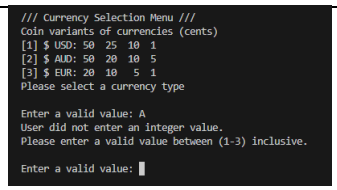
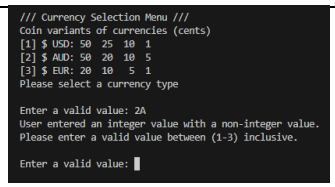
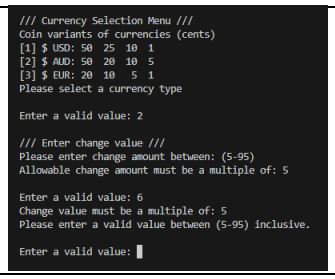
File name	Name of function/subroutine in file	Purpose of the function/subroutine	Single responsibility? (Y/N)
a1_coins_main.c	main()	Controls the program flow. It calls other modules from a top-down sequence inside of a do-while loop until the user chooses to quit	Yes
a1_coins_func.c	getUserInt()	Gets a valid integer input within a min-max range inside of a do-while loop. Calls the validateUserInput function to validate the input. Re-prompts the user to try again if its invalid.	Yes
-	validateUserInput()	Validate the user's input, displaying error messages if any conditions are met. Returns 1 or 0. Depending on error, may call clearInputBuffer function	Yes
-	clearInputBuffer()	Clears the input buffer removing the remaining characters if there are any.	Yes
-	getCoinValue()	Returns an int value that is dependent on the condition of the userCurrencyType integer	Yes
	getCoinAmount()	Returns the number of coins required after making an integer division calculation.	yes
	getChangeRemaining()	Returns an int value after making an integer subtraction calculation.	yes
-	displayUserValues()	Displays the inputs the user has entered before displaying the sorted coins.	Yes
-	displayCoins()	Displays the number required of each coin that sum to the equivalent of the change the user had entered.	Yes
-	promptUserCurrency()	Displays the currency choice options to the user.	Yes
-	promptUserChange()	Displays the range and a specified multiple to the user.	yes
-	promptUserExit()	Displays the exit menu options to the user.	yes
-	pauseExitProgram()	Displays final instruction, awaits user to press enter to quit.	yes

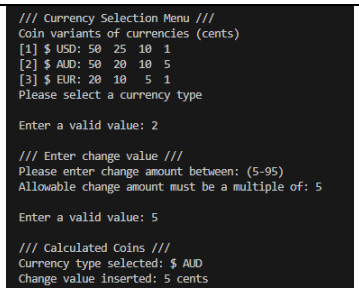
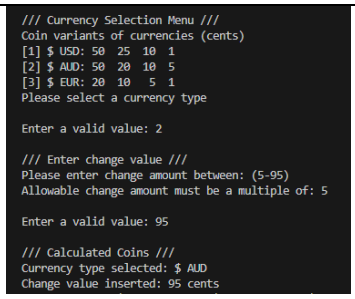
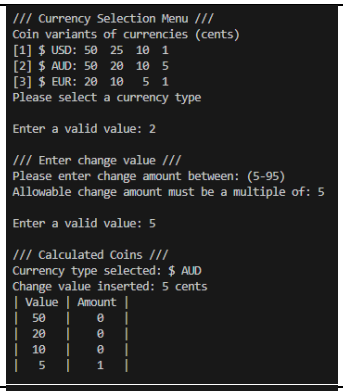
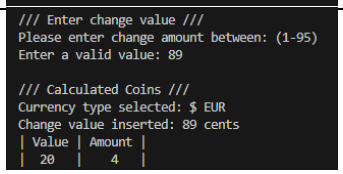
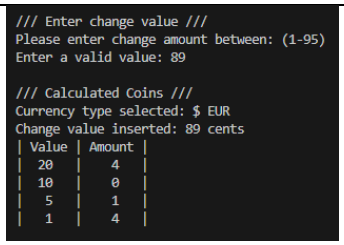
6. Program Testing (10%)

Provide results of applying your test data to your final program (tabular form), including screen shots of your program in operation. Add rows to the following table as needed. The table can span more than one page if needed.

The table is a copy/paste of the desk check, except the actual output column shows the results of the actual program output. There should be no duplicated reasons listed in the second column.

No marks will be awarded to this section if your code (program) does not build and run.

Test id	Test description/justification – what is the test for and why this particular test.	Actual data for this test	Expected output	Actual program output when test is carried out	Program test outcome – Pass/Fail
1	Testing lower boundary violation for currency selection. A value of 0 will be entered as it's below the minimum range. Why: A user may accidentally enter the value.	Module: validateUserInput valid = 1 userInput: 0 Rangemin: 1 RangeMax: 3	Print: "User entered a value outside of range" return value 0;		Pass
2	Testing upper boundary violation for currency selection. A value of 4 will be entered as it's below the minimum range. Why: A user may accidentally enter the value.	Module: validateUserInput valid = 1 userInput: 4 Rangemin: 1 RangeMax: 3	Print: "User entered a value outside of range" return value 0;		Pass
3	Testing data type non-numeric violation for currency selection. A value of 'A' will be entered as it's a char value. Why: A user may accidentally enter the value, checking the if !invalid) branch	Module: validateUserInput valid = 0 userInput: 'A' Rangemin: 1 RangeMax: 3	Print: "User entered a non-integer value" clearInputBuffer() return value 0;		Pass
4	Testing a valid value joined with a non-numeric value for currency selection. A value of '2A' will be entered. Why: A user may accidentally enter the value along with their intentional valid value. Doing so will check the If (valid && getChar() != '\n') branch	Module: validateUserInput valid = 1 userInput: '2A' Rangemin: 1 RangeMax: 3	Print: "User entered an integer with a non-integer value." clearInputBuffer() return value 0;		Pass
5	Insert a valid value for AUD change amount that is NOT a multiple of 5 A value of '6' will be entered as it's technically a valid integer value within the range. Why: Testing an edge case scenario where the user may enter a value that is not a multiple of 5 for AUD.	Module: validateUserInput valid = 1 userInput: 6 Rangemin: 5 RangeMax: 95	Print: "Change value must be a multiple of 5" return value 0;		Pass

6	Testing the lower boundary for AUD change amount. A value of '5' will be entered as it's a valid integer value. Why: Testing best case scenario of a user entering the minimum valid value that will pass the IF branch filter. Doing so should return a value back to main.	Module: getUserInput Rangemin: 5 RangeMax: 95 userInput: 5 valid = 1	Returns userInput value: 5		Pass
7	Testing the upper boundary for AUD change amount. A value of '95' will be entered as it's a valid integer value. Why: Testing best case scenario of a user entering the maximum valid value that will pass the IF branch filter. Doing so should return a value back to main.	Module: getUserInput Rangemin: 5 RangeMax: 95 userInput: 95 valid = 1	Returns userInput value: 95		Pass
8	Test value allocation amount A currencytype of 2 will be entered for inserting a value for the first coin. Why: Testing a scenario that verifies that the correct value is inserted for the first coin. If that works, then the rest should work.	Module: getCoinValue currencyType: 2 coinVal_1: 50 coinVal_2: 50 coinVal_3: 20	Returns coinVal_2 value: 50		Pass
9	Test getting the correct coin amount. Using currencyType 3, we will use the first coin and find the appropriate amount coins to allocate for the userChange of 89. Why: Because we want the highest number of coins for each coin value from highest to lowest.	Module: getCoinAmount currencyType: 3 coinValue: 20 userChange: 89	Returns coinAmount value: 4		Pass
10	Test the remaining change after calculation. using currencyType 3, number of coins 4, a coin value of 20 and user's change of 89. Why: We need to make sure that the correct change is left over after finding subtracting the coin amount multiplied by the coin value.	Module: getChangeRemaining currencyType: 3 coinValue: 20 coinAmount: 4 userChange: 89	Returns changeRem value: 9	 5 * 1 = 5 1 * 4 = 4 5 + 4 = 9	Pass

7. Self-Assessment (5%)

7.1. Self-assessment of how successful you were in achieving the requirements and a discussion of any problems you encountered. **(2.5%)**

I believe I successfully met the Minimum Viable Product requirements for this assignment. The final program displays the correct change amount using the appropriate number of coin values based upon the change value the user inserted. As the program's main stack progresses it accepts an integer currency type selection that is within a valid range (1-3), an integer change value within a valid range of (1-95), and an integer quit/retry value within a valid range (1-2). The program design had considered some various user input errors, should an error occur, it should provide the user with the appropriate error message, then re-prompt the user to try again.

Having previously completed a similar exercise for Harvard's CS50 course many years ago, I had some prior knowledge of what to expect, the core logic, and the user errors that are likely show up for this program. Initially, I found it significantly easier to figure out what the program would need by designing my own UML Activity Diagram (which - I know I was not required to do). I think this assignment follows very closely to the Waterfall SDLC method, and so to make any changes at all, it has been an extremely costly process of having to go through and edit the entire documentation.

Core logic of pseudocode is retained, with very minor cosmetic deviations for the final product:

- *added #define constants replacing the various highlighted magic numbers throughout my pseudocode and final code (a1_coins_func.h – Line: 12 to 33)*
- *corrected grammar for several print statement messages and improved the clarity of each message (a1_coins_func.c – Line: various).*

Some problems I encountered:

- *I tried implementing a screen wipe that James showed me (but it did not work in CMD, only in the VS Code terminal):*

```
// printf("\033[2J\033[H");
```

- *Trying to figure out if I could use the same getUserInput() function reliably for all my get input functions for picking currency, entering a change value, and quitting the program.*
- *Trying to solve the validation function of the user input, there was a unique specification for AUD where the value had to be a multiple of 5, while a solution was found, I think it could be considered slightly "hacky."*

I could improve my program, solution further by:

- *incorporation of using arrays/structs/typedefs for the currency arrays so that the size of the array and its type as a string is always retrievable without having to use sizeOf()/defined constants or an array of strings. It would be a lot cleaner to look at than repeated function calls (though out of scope of project requirements)*
- *use fgets instead of scanf for the getUserInput for better filtering, having a buffer that accepts two values + the null terminator, then type casting the received char/s to an integer value.*
- *I could merge the getCoinAmount and getChangeRemaining steps by using a pointer to userChange, performing both the div calculation and the subtraction calculation BUT it might not adhere to the high cohesion guidelines of this unit.*
- *I could potentially store all the print statements into a string array, using a single function to display each message, possibly using an ENUM for each display/prompt for the user. (Again, out of scope and beyond what the assignment asks for).*
- *Reduce the "hacky-ness" of the filtering function, I think the result was more convoluted/verbose than it had to be.*
- *Clarifying the getCoinValue function's parameter names for the coinsVals_x, I should have renamed them to usdVal, audVal, eurVal for readability in hindsight.*

7.2. In point form, a summary of what works and what does not work in your program. A false claim here would mean that more than 80% of the marks for the whole assignment would not be awarded. So, make sure that you have tested your program thoroughly. **(2.5%)**

What works for the program:

- *Accepts an input within range for a user to choose a currency type*
- *Accepts an input within range for a user to insert a change amount*
- *Accepts an input within range for a user to choose to retry/quit the program*
- *Displays an adequate error message if caught (though there may be possible unknown unknowns.)*
- *Displays a currency menu prompt*
- *Displays a user change prompt*
- *Displays an exit program prompt*
- *Displays the correct coin amount per coin value.*
- *Compiles without errors/warnings*

What does not work for the program:

- *As far as I'm aware, the program works.*
- *I've been unable to run into any issues outside of the scope for this assignment.*